Ain Shams University, Faculty of Engineering

Web Development
Todo App Lab Task

Group 1 Section 1
Mohamed Ashraf Hatem Eltaie 24p0024

Todo App.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Todo App</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="todo-container">
        <h1>Todo App</h1>

        <!-- Input Form -->
        <div class="todo-input-group">
            <input type="text" id="todo-input" placeholder="What needs doing?" autofocus>
            <button id="add-btn">Add</button>
        </div>

        <!-- Controls / Filters bar -->
        <div class="controls-bar">
            <span class="tip">Tip: double-click a todo to edit. Press Enter to save.</span>
            <div class="filters">
                <button class="filter-btn active" data-filter="all">All</button>
                <button class="filter-btn" data-filter="active">Active</button>
                <button class="filter-btn" data-filter="completed">Completed</button>
            </div>
        </div>

        <!-- Todo List -->
        <ul id="todo-list">
            <!-- Dynamic items will be injected here -->
        </ul>

        <!-- Footer / Summary bar -->
        <div class="todo-footer">
            <span id="items-left">0 items left</span>
            <div class="footer-actions">
                <button id="clear-completed">Clear completed</button>
                <button id="clear-all">Clear all</button>
            </div>
        </div>
    </div>

    <!-- Link the compiled JavaScript file -->
    <script src="client.js" type="module"></script>
</body>
</html>
```

style.css

```
body {
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-
serif;
    background-color: #f5f5f5;
    color: #4d4d4d;
    display: flex;
    justify-content: center;
    align-items: flex-start;
    padding-top: 50px;
    margin: 0;
    height: 100vh;
}

.todo-container {
    background: #fff;
    width: 600px;
    padding: 20px 30px;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.2);
    border-radius: 4px;
}

h1 {
    font-size: 28px;
    color: #2c3e50;
    margin-top: 0;
    margin-bottom: 20px;
}

.todo-input-group {
    display: flex;
    margin-bottom: 15px;
}

#todo-input {
    flex: 1;
    padding: 12px 15px;
    font-size: 16px;
    border: 1px solid #ddd;
    border-radius: 4px 0 0 4px;
    outline: none;
}

#todo-input:focus {
    border-color: #3b82f6;
}

#add-btn {
    padding: 0 20px;
    background-color: #3b82f6;
    color: white;
    font-size: 16px;
    border: none;
    border-radius: 0 4px 4px 0;
    cursor: pointer;
}
```

```css
#add-btn:hover {
    background-color: #2563eb;
}

.controls-bar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 13px;
    color: #777;
    margin-bottom: 15px;
    border-bottom: 1px solid #eee;
    padding-bottom: 10px;
}

.filters button {
    background: none;
    border: none;
    cursor: pointer;
    padding: 4px 8px;
    margin-left: 4px;
    border-radius: 4px;
    color: #555;
    font-size: 13px;
}

.filters button.active {
    background-color: #eff6ff;
    color: #3b82f6;
    font-weight: bold;
}

#todo-list {
    list-style: none;
    padding: 0;
    margin: 0 0 15px 0;
}

.todo-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 12px 10px;
    border-bottom: 1px solid #eee;
    font-size: 16px;
}

.todo-item.completed .todo-text {
    text-decoration: line-through;
    color: #d9d9d9;
}

.todo-left {
    display: flex;
    align-items: center;
    gap: 12px;
    flex: 1;
}
```

```css
.todo-checkbox {
    width: 20px;
    height: 20px;
    cursor: pointer;
}

.delete-btn {
    background: none;
    border: none;
    cursor: pointer;
    color: #999;
    font-size: 16px;
}

.delete-btn:hover {
    color: #ef4444;
}

.todo-footer {
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 14px;
    color: #777;
    border-top: 1px solid #eee;
    padding-top: 15px;
}

.footer-actions button {
    background: none;
    border: 1px solid #ddd;
    border-radius: 4px;
    padding: 6px 12px;
    cursor: pointer;
    color: #555;
    font-size: 13px;
    margin-left: 5px;
}

.footer-actions button:hover {
    background-color: #f9f9f9;
    border-color: #999;
}
```

client.ts

```ts
// ========================================
// 1. Interfaces & Types
// ========================================
class Todo {
    private static currentID: number = 0;
    public readonly id: number;
    public text: string;
    public completed: boolean;

    constructor(text: string) {
        this.id = Todo.currentID;
        Todo.currentID++;
        this.text = text;
        this.completed = false;
    }
}

enum FilterType {
    All = 'all',
    Active = 'active',
    Completed = 'completed'
}

// ========================================
// 2. Global State Variables
// ========================================
let todos: Todo[] = [];
let currentFilter: FilterType = FilterType.All;

// ========================================
// 3. DOM Element Selectors
// ========================================
const todoInput = document.getElementById('todo-input') as HTMLInputElement;
const addBtn = document.getElementById('add-btn') as HTMLButtonElement;
const todoList = document.getElementById('todo-list') as HTMLUListElement;
const itemsLeft = document.getElementById('items-left') as HTMLSpanElement;
const filterBtns = document.querySelectorAll('.filter-btn');
const clearCompletedBtn = document.getElementById('clear-completed') as HTMLButtonElement;
const clearAllBtn = document.getElementById('clear-all') as HTMLButtonElement;

// ========================================
// 4. Core Functions (Implement these!)
// ========================================

/**
 * Reads the input value, creates a new Todo object, pushes it to the array,
 * clears the input field, and triggers a re-render.
 */
function AddTodo(): void {
    const text = todoInput.value.trim();
    if (text === '') return; // Don't add empty todos

    const todo: Todo = new Todo(text);
    todos.push(todo);
    todoInput.value = '';
    Render();
```

```
}

/**
 * Finds a todo by its ID and flips its 'completed' status, then re-renders.
 */
function ToggleTodo(id: number): void {
    const todo: Todo | undefined = todos.find(item => item.id === id);
    if(todo === undefined){
        return;
    }
    todo.completed = !todo.completed;
    Render();
}

/**
 * Removes a todo item from the todos array by its ID, then re-renders.
 */
function DeleteTodo(id: number): void {
    todos = todos.filter(item => item.id !== id);
    Render();
}

/**
 * Updates the text of a specific todo item by its ID, then re-renders.
 */
function EditTodo(id: number, newText: string): void {
    const todo: Todo | undefined = todos.find(item => item.id === id);
    if(todo === undefined){
        return;
    }
    todo.text = newText;
    Render();
}

/**
 * Clears completed todos from the array, then re-renders.
 */
function ClearCompleted(): void {
    todos = todos.filter(item => !item.completed);
    Render();
}

/**
 * Clears all todos from the array, then re-renders.
 */
function ClearAll(): void {
    todos.length = 0;
    Render();
}

/**
 * Clears the HTML list, filters the todos based on currentFilter,
 * generates the HTML elements, and updates the items-left counter.
 */
function Render(): void {
    // 1. Clear the existing HTML list to prevent duplicates
    todoList.innerHTML = '';

    // 2. Filter the todos array based on the selected filter tab ('all', 'active', 'completed')
```

```
    const filteredTodos = todos.filter(todo => {
        if (currentFilter === FilterType.Active) return !todo.completed;
        if (currentFilter === FilterType.Completed) return todo.completed;
        return true; // 'all'
    });

    // 3. Loop through the filtered list and build DOM elements
    filteredTodos.forEach(todo => {
        const li = document.createElement('li');
        // Add the 'completed' class if the todo is finished (applies line-through style)
        li.className = `todo-item ${todo.completed ? 'completed' : ''}`;

        // Construct the inner HTML structure, passing the item's unique ID to data attributes
        li.innerHTML = `
            <div class="todo-left">
                <input type="checkbox" class="todo-checkbox" ${todo.completed ? 'checked' : ''}
data-id="${todo.id}">
                <span class="todo-text" data-id="${todo.id}">${todo.text}</span>
            </div>
            <button class="delete-btn" data-id="${todo.id}">🗑</button>
        `;

        // Append the newly created <li> item to the <ul> list
        todoList.appendChild(li);
    });

    // 4. Calculate how many items are left (incomplete)
    const activeCount = todos.filter(todo => !todo.completed).length;

    // 5. Update the footer counter text
    itemsLeft.textContent = `${activeCount} item${activeCount === 1 ? '' : 's'} left`;
}

// ========================================
// 5. Event Listeners Setup
// ========================================

// Listen for click on Add button
addBtn.addEventListener('click', () => {
    AddTodo();
});

// Listen for Enter key on the input field
todoInput.addEventListener('keydown', (e: KeyboardEvent) => {
    if (e.key === 'Enter') {
        AddTodo();
    }
});

// Event delegation on the todo list for checkboxes, delete buttons, and editing
todoList.addEventListener('click', (e: MouseEvent) => {
    const target = e.target as HTMLElement;

    // Get the data-id from the clicked element
    const idAttr = target.getAttribute('data-id');
    if (!idAttr) return; // Exit if the clicked element doesn't have a data-id

    const id = Number(idAttr);
    if (target.classList.contains('todo-checkbox')) {
```

```
        ToggleTodo(id);
    }
    else if (target.classList.contains('delete-btn')) {
        DeleteTodo(id);
    }
});

// Event delegation on the todo list for inline editing
todoList.addEventListener('dblclick', (e: MouseEvent) => {
    const target = e.target as HTMLElement;

    if (target.classList.contains('todo-text')) {
        const idAttr = target.getAttribute('data-id');
        if (!idAttr) return;
        const id = Number(idAttr);

        const currentText = target.textContent || '';

        // Create a temporary input element for editing
        const input = document.createElement('input');
        input.type = 'text';
        input.value = currentText;
        input.className = 'edit-input';
        input.style.flex = '1';
        input.style.padding = '6px';
        input.style.fontSize = '16px';
        input.style.border = '1px solid #3b82f6';
        input.style.borderRadius = '4px';
        input.style.outline = 'none';

        // Replace the text span with the input box
        target.replaceWith(input);
        input.focus();

        // Helper function to save changes
        const save = () => {
            const newText = input.value.trim();
            if (newText !== '') {
                EditTodo(id, newText);
            } else {
                Render(); // Revert if empty
            }
        };

        // Save on Enter or cancel/revert on Escape
        input.addEventListener('keydown', (ke: KeyboardEvent) => {
            if (ke.key === 'Enter') {
                save();
            } else if (ke.key === 'Escape') {
                Render();
            }
        });

        // Also save when clicking outside the input box (blur)
        input.addEventListener('blur', () => {
            save();
        });
    }
});
```

```
// Listen for clicks on filter buttons (All, Active, Completed)
filterBtns.forEach(btn => {
    btn.addEventListener('click', (e) => {
        const target = e.currentTarget as HTMLElement;
        const filterValue = target.getAttribute('data-filter');

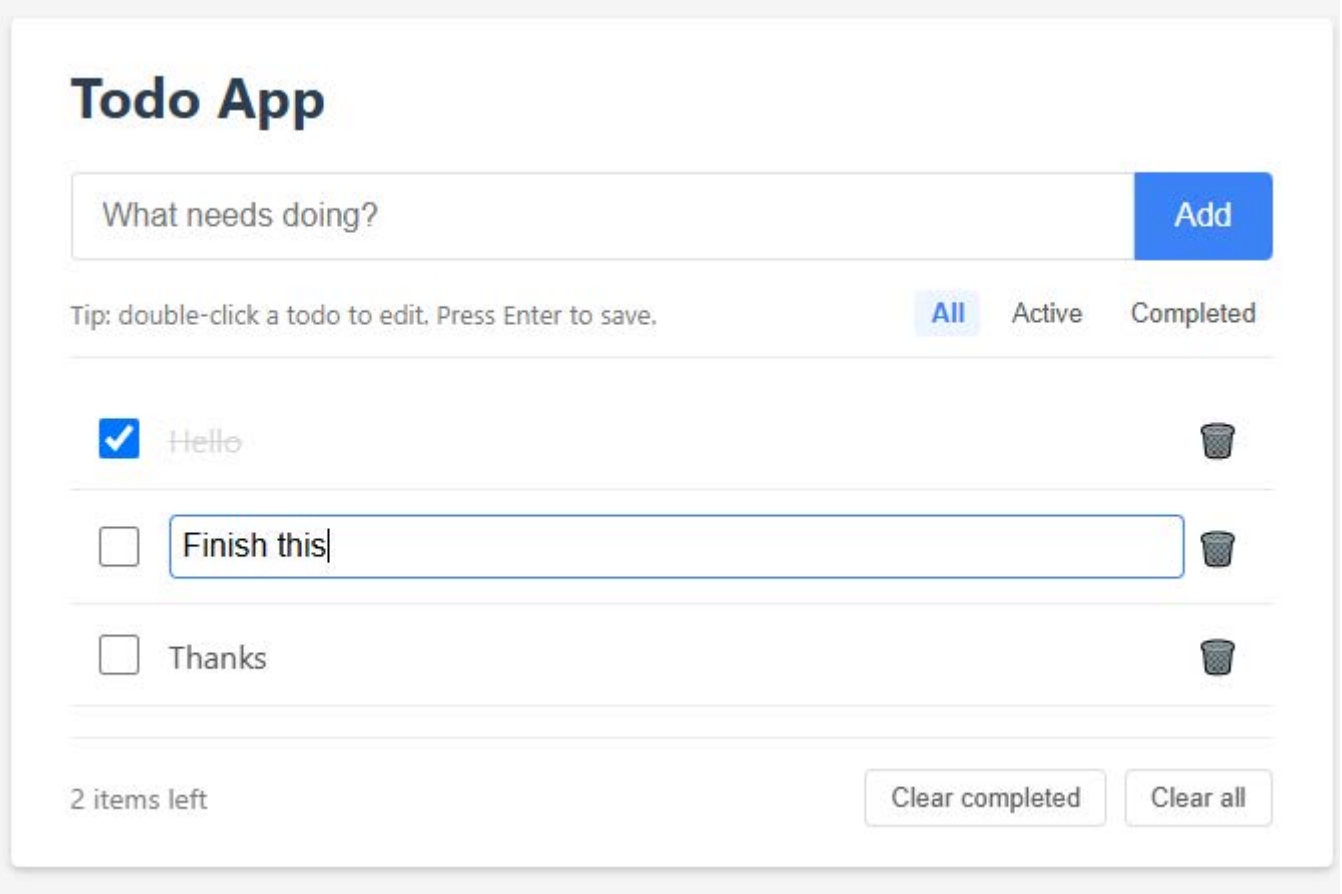
        if (filterValue) {
            filterBtns.forEach(b => b.classList.remove('active'));
            target.classList.add('active');
            currentFilter = filterValue as FilterType;
            Render();
        }
    });
});

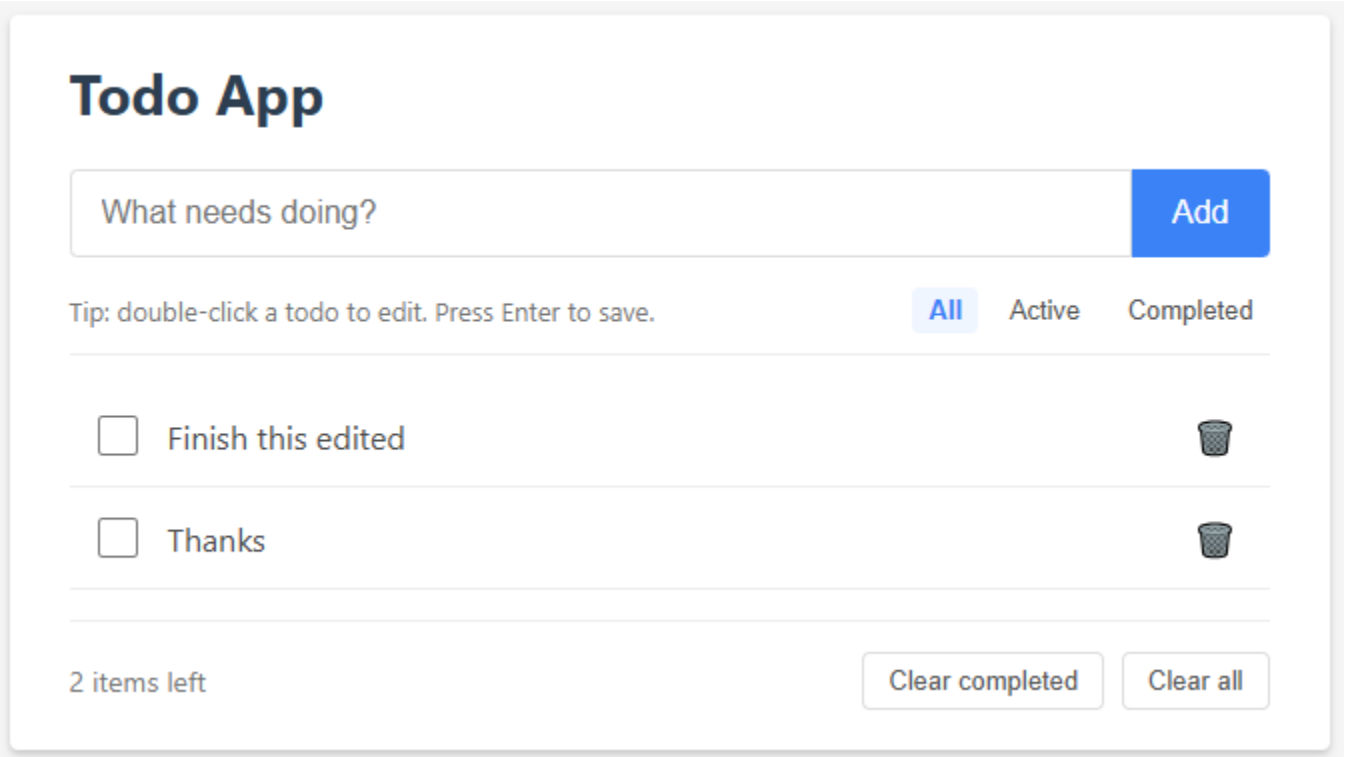
// Listen for click on "Clear completed"
clearCompletedBtn.addEventListener('click', () => {
    ClearCompleted();
});

// Listen for click on "Clear all"
clearAllBtn.addEventListener('click', () => {
    ClearAll();
});

// Initial render call when the script loads
Render();
```

Screenshot


Todo App
What needs doing?
Add
Tip: double-click a todo to edit. Press Enter to save.
All
Active
Completed
Hello
Finish this
Thanks
2 items left
Clear completed
Clear all


Todo App
What needs doing?
Add
Tip: double-click a todo to edit. Press Enter to save.
All
Active
Completed
Finish this edited
Thanks
2 items left
Clear completed
Clear all

Github Link

https://github.com/ManCityP/Web_Development_CSE343